Governed Semantic Operation Language

A Constitutional Interface Between Prose, Policy, Code, Proof, and Execution

When Pseudocode Falls Short

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Computational systems increasingly perform actions whose significance cannot be determined from execution alone: a program may run, a model may answer, a tool may return output, or an artifact may be built without establishing whether the transition was authorized, evidenced, bounded, or admissible. This paper introduces the Governed Semantic Operation Language, or GSOL, as a specification-layer language for representing such operations as admissible semantic transitions before execution, lowering, or evidentiary use. GSOL's contribution is a canonical operation form that makes authority, effect, refusal, receipt, claim, nonclaim, standing, temporal validity, and lowering obligations explicit. GSOL is not a runtime, truth engine, policy engine, or replacement for compiler intermediate representations; it is a governance layer for preserving admissibility-relevant meaning across specification and realization. The result is a conceptual framework for a language of admissible operation: expressive enough to govern meaningful computational transitions, but deliberately not so computationally powerful that it becomes an opaque execution substrate.

Keywords

governed computation; semantic operation language; admissibility; intermediate representation; constitutional AI; typed claims; refusal semantics; receipts; provenance; authority; standing; model governance; tool governance; memory governance; lowering; semantic hashing

1. Introduction

Modern computational systems increasingly perform actions whose significance cannot be understood from execution alone. A program may run, a model may answer, a tool may return output, a build may pass, or an artifact may be packaged, while the surrounding system remains silent about whether the

transition was authorized, what evidence supports it, what effects it had, what claims may be drawn from it, and what limits must be preserved.

The missing distinction is between capability and admissibility. Capability asks whether a system can perform an operation. Admissibility asks whether the operation may be treated as a governed transition under a specified authority, jurisdiction, evidence surface, and standing rule. A capable system can still perform an unauthorized action. A successful run can still fail to support the claim later attached to it. A model output can be fluent while remaining mere testimony. A receipt can be internally consistent while failing to establish external provenance.

This paper introduces GSOL, the Governed Semantic Operation Language. GSOL is a language for representing operations as governed semantic transitions. By governed semantic transition, this paper means an operation whose admissibility depends on explicit authority, effect, evidence, refusal, claim, nonclaim, standing, temporal validity, or lowering conditions. By admissibility-relevant meaning, this paper means what the operation is allowed to do, what evidence it must produce, what it may support, and what it must not imply.

GSOL is not proposed as a general-purpose programming language. It is not a replacement for compiler intermediate representations, policy engines, provenance systems, model runtimes, or tool harnesses. Instead, it is a missing middle layer between governed prose and executable realization. It gives admissibility-relevant meaning a code-like form before that meaning is translated into code, intermediate representation, runtime behavior, or evidence records.

The core contribution of this paper is the definition of GSOL as a canonical language of admissible operation: a representation for computational transitions that must preserve authority, effect, refusal, receipt, claim, nonclaim, standing, temporal validity, and lowering boundaries.

The specific contributions are:

1. A definition of GSOL as a governed semantic operation language.
2. A formal transition model for admission, refusal, and gap outcomes.
3. A minimal distinction among testimony, evidence, typed claims, authority, receipts, and standing.
4. A threat model separating internal consistency from external attestation.
5. A worked end-to-end example from proposal through lowering.
6. A comparison with intermediate representations, policy languages, provenance systems, and AI harness layers.
7. A conformance boundary for future parsers, checkers, runtimes, and lowering systems.

The central thesis is simple: governed computation needs a language of admissible operation. Such a language should be expressive enough to preserve admissibility-relevant meaning, but not so computationally expressive that it becomes the

execution substrate it was meant to govern.

2. Motivation

Many failures in governed computation arise from category collapse. A system treats one class of object as if it were another: testimony as truth, evidence as authority, execution as standing, policy as receipt, or provenance as semantic preservation.

The motivating distinctions can be stated without relying on a wide table:

Capability. Capability means the system can perform an operation. GSOL prevents capability from being treated as authority.

Authority. Authority means a warrant or policy admits an operation under jurisdiction. GSOL prevents identity, availability, or success from being treated as permission.

Evidence. Evidence is a record that may support a typed claim. GSOL prevents evidence from being treated as self-interpreting standing.

Standing. Standing is an admitted level of evidentiary support. GSOL prevents fixture success from being promoted into release or production readiness without an admitted standing transition.

Receipt. A receipt is a bound record of a transition, refusal, gap, or standing change. GSOL prevents receipt presence from being treated as provenance or truth by itself.

Claim. A claim is a typed assertion admitted under an entailment rule. GSOL prevents natural-language prose from being treated as machine-checkable truth.

Nonclaim. A nonclaim is a forbidden inference preserved beside evidence. GSOL prevents evidence from being used to imply stronger conclusions than it supports.

Refusal. Refusal is a governed result that blocks an inadmissible transition. GSOL treats refusal as successful boundary preservation rather than mere error.

These distinctions matter in ordinary software, but they become especially important in AI-in-the-loop systems. A governed AI environment may receive a request, call a model, classify output, call tools, write memory, produce a response, and attach evidence. Without a semantic operation layer, each transition risks silent overpromotion.

GSOL addresses this by forcing each governed operation to declare its authority, effect, refusal paths, receipts, claims, nonclaims, standing limits, and lowering status.

3. Related Work

GSOL is adjacent to several existing traditions but has a different center of gravity.

3.1 Compiler intermediate representations

Compiler intermediate representations such as LLVM IR provide structured representations for program analysis, transformation, optimization, and code generation. LLVM IR, for example, is a static single-assignment based representation used as a common code representation across compiler phases. GSOL is not a competitor to such IRs. Conventional IRs are designed to preserve computational behavior. GSOL is designed to preserve admissibility-relevant meaning: authority, effect, refusal, receipt, claim, nonclaim, standing, temporal validity, and lowering obligations.

A compiler IR may faithfully lower a program while dropping the reason the operation was admitted, the evidence it must emit, or the nonclaims that constrain interpretation. GSOL is intended to sit above or beside such IRs so that lowering into an executable substrate remains semantically honest.

3.2 Policy languages and access-control systems

Policy languages decide whether actions are permitted under rules. They are necessary but not sufficient for GSOL's purpose. GSOL includes authority and policy checks, but also binds receipts, claims, nonclaims, standing, lowering, temporal validity, and gap certificates. A policy language may answer whether a request is allowed. GSOL represents the whole governed transition: proposal, admission or refusal, effect, evidence, claim boundary, nonclaim boundary, and replay surface.

3.3 Provenance systems

Provenance systems describe where artifacts, data, or activities came from. W3C PROV, for example, provides a vocabulary for representing provenance information across systems. GSOL is compatible with provenance systems but does not reduce to them. Provenance can say how an artifact was produced. GSOL also asks whether the operation was authorized, what effect class was admitted, what refusal paths existed, what claims the evidence supports, and what nonclaims remain binding.

3.4 Software supply-chain attestation

Software supply-chain attestation systems such as in-toto and SLSA focus on artifact integrity, build provenance, and protection against tampering across production chains. These systems are close cousins for release and artifact integrity. GSOL complements them by specifying the semantic operation whose execution, build, release, or attestation is being governed. A GSOL receipt

may be exported into in-toto-style or SLSA-style provenance, but GSOL also preserves claim and nonclaim boundaries that ordinary supply-chain attestations may not express.

3.5 Runtime harnesses and AI governance layers

AI harnesses and agent frameworks orchestrate model calls, tool calls, memory, and responses. GSOL does not replace such harnesses. It gives them a canonical operation language. A governed AI harness can use GSOL to state that a model invocation requires a warrant, that model output is testimony rather than truth, that a tool request requires effect admission, and that memory writes require separate admission.

3.6 Controlled natural and machine languages

Controlled natural languages reduce ambiguity in human-readable statements. Machine-oriented controlled languages reduce ambiguity in machine-facing procedures. GSOL occupies the operational middle: it is more structured than governed prose but does not claim to be the final executable substrate. It stabilizes operations rather than ordinary sentences or low-level instructions.

4. Scope and Non-Goals

GSOL governs semantic transitions. It does not govern everything.

GSOL does not decide every question of truth. It does not replace human judgment, external law, cryptographic attestation, production monitoring, or runtime security. It does not make unsafe code safe merely by describing it. It does not make a model reliable merely by classifying its output. It does not prove arbitrary natural-language claims. It does not execute binaries.

GSOL is not:

```
a Turing-complete language
a general-purpose programming language
a replacement for MCIR or Jnana
a replacement for Python or native code
a shell scripting language
a live policy engine by itself
a complete entailment engine by itself
a model runtime
a tool runtime
a memory database
a production attestation service
a natural-language truth engine
```

These are design boundaries, not disclaimers of failure. A language intended to govern execution should not silently become the execution substrate. GSOL's

target is constitutional adequacy, not computational universality.

5. Constitutional Kernel

GSOL is governed by a compact kernel of invariants. They are grouped here by class.

Authority and effect.

```
Authority before capability.
Warrant before effect.
Effect before action.
Authority is external to assertion.
Tool availability is not tool authority.
```

Evidence and receipts.

```
Receipt before effect.
Evidence is not authority.
Determinism or declared nondeterminism.
Currency comes from current head.
```

Claims and standing.

```
Testimony is not claim admission.
Claims require typed admission.
Standing is never implied.
Nonclaims block stronger claims.
```

Temporal and lowering discipline.

```
No silent semantic promotion.
Correspondence is not equivalence.
Refusal is successful governance.
Memory writes require admission.
Gap before silent loss.
```

The kernel is the paper's compact thesis. GSOL exists to prevent these categories from collapsing. Later sections define how operations preserve the kernel through typed fields, transition rules, receipts, refusal paths, and lowering declarations.

6. Formal Model

A GSOL program is a finite set of governed operation declarations plus metadata and registry bindings.

$$P = (M, O, B)$$

where:

```
M = metadata, imports, profile, and current-head references
O = a finite set of operation declarations
B = registry bindings
```

A GSOL operation has the canonical field order:

$$\begin{aligned} O_i = (&\mathrm{id}, \mathrm{ver}, \mathrm{jur}, \mathrm{prof}, \mathrm{in}, \\ &\mathrm{out}, \mathrm{auth}, \mathrm{warr}, \mathrm{eff}, \mathrm{req}, \\ &\mathrm{ref}, \mathrm{adm}, \mathrm{emit}, \mathrm{clm}, \mathrm{nclm}, \\ &\mathrm{st}, \mathrm{tmp}, \mathrm{debt}, \mathrm{low}, \mathrm{proof}) \end{aligned}$$

where:

```
id    = operation identifier
ver   = operation version
jur   = jurisdiction
prof  = profile
in    = inputs
out   = outputs
auth  = authority requirements
warr  = warrants
eff   = effects
req   = preconditions
ref   = refusal rules
adm   = admitted action
emit  = receipt obligations
clm   = typed claims supported
nclm  = nonclaims preserved
st    = standing bounds
tmp   = temporal validity constraints
debt  = semantic debt
low   = lowering declarations
proof = proof obligations
```

A GSOL operation denotes a partial transition function:

$$T_i : S \times I \times A \to \mathrm{Res}$$

where S is governed state, I is input, A is authority context, and Res is a result.

The result type is:

$$\begin{aligned} \mathrm{Res} ::= &\ \mathrm{Admit}(S', Y, \mathcal{R}, \Delta) \\ &\mid \mathrm{Refuse}(c, \mathcal{R}, \Delta) \\ &\mid \mathrm{Gap}(g) \end{aligned}$$

where:

```
S'          = next state
Y           = output
mathcal R   = receipt set
Delta       = standing delta
c           = refusal code
g           = gap certificate
```

Admission is defined as:

$$\begin{aligned}\text{Admissible}(O_i, S, I, A) \Leftrightarrow \text{Req}(O_i, S, I, A) \\ \wedge\ \text{Auth}(O_i, A) \\ \wedge\ \text{Warr}(O_i, A) \\ \wedge\ \text{Eff}(O_i, A) \\ \wedge\ \text{Emit}(O_i) \\ \wedge\ \text{Stand}(O_i) \\ \wedge\ \text{Nonclaim}(O_i) \\ \wedge\ \text{Temp}(O_i)\end{aligned}$$

This relation does not execute the operation. It determines whether the operation is constitutionally admitted.

7. Formal Example: Transition and Refusal

Consider a local write operation O_w that writes a canonical value to a governed store.

Let the authority context A contain a warrant w. The operation requests effect `WRITE_LOCAL`. The core admission rule is:

$$\frac{\begin{array}{c} w \in A.\text{warrants} \quad \text{permits}(w, \text{WRITE_LOCAL}) \quad \text{active}(w) \quad \text{canonical}(\text{value}) \\ \text{effect}(O_w) \leq \text{permitted}(w) \quad \text{receipt_declared}(O_w, \text{pre_effect}) \quad \text{receipt_declared}(O_w, \text{closeout}) \end{array}}{T_w(S, I, A) = \text{Admit}(S', y, \mathcal{R}_{\text{admit}}, \Delta)}$$

A refusal rule for a missing warrant is:

$$\frac{\neg \exists w \in A.\text{warrants} : \text{permits}(w, \text{WRITE_LOCAL}) \quad \text{receipt_declared}(O_w, \text{refusal})}{T_w(S, I, A) = \text{Refuse}(\text{REFUSE.MISSING_WARRANT}, \mathcal{R}_{\text{refusal}}, \Delta_0)}$$

A refusal rule for stale authority is:

$$\frac{\exists w \in A.\text{warrants} : \text{permits}(w, \text{WRITE_LOCAL}) \quad \neg\text{active}(w) \quad \text{receipt_declared}(O_w, \text{refusal})}{T_w(S, I, A) = \text{Refuse}(\text{REFUSE.STALE_WARRANT}, \mathcal{R}_{\text{refusal}}, \Delta_0)}$$

The important property is not the complexity of the example. It is the ordering: the effect is not performed first and justified later. Admission or refusal precedes the effect, and every refusal path has a corresponding refusal receipt obligation.

8. Canonical Object Model

This section defines the core objects used by GSOL. These definitions are canonical within this paper; later sections use these meanings without redefining them.

8.1 State

State is the governed substrate before and after an operation.

```
State =
  (state_id,
   state_hash,
   content_hash,
   semantic_hash,
   jurisdiction_hash,
   registry_refs,
   temporal_status,
   lineage_refs)
```

State is not merely memory. It is an evidence-bearing surface with lineage, hashes, and admissibility.

8.2 Authority and Warrant

Authority is the permission structure that admits or refuses an operation.

```
AuthorityContext =
  (actor_ref,
   warrants,
   roles,
   policies,
   jurisdiction,
   delegation_chain,
   revocation_status,
   freshness,
   trust_anchor_refs)
```

A warrant is a bounded authority object.

```
Warrant =
  (warrant_id,
   actor_ref,
   jurisdiction_ref,
   permitted_effects,
   permitted_targets,
   profile_ref,
   valid_from,
   valid_until,
   revocation_ref,
   receipt_obligations)
```

Authority is distinct from identity, capability, receipt, and standing.

```
Identity != Authority
Capability != Authority
Receipt != Authority
SuccessfulRun != Standing
```

8.3 Effect

An effect is the class of internal or external change an operation may cause.

```
READ_ONLY
WRITE_LOCAL
WRITE_EXTERNAL
MUTATE_STATE
MODEL_INVOKE
TOOL_CALL
MEMORY_WRITE
CLAIM_ADMIT
CLAIM_RETIRE
ISSUE_AUTHORITY
REVOKE_AUTHORITY
PUBLISH
NETWORK
EXECUTE_PROCESS
CROSS_JURISDICTION
MERGE_LINEAGE
ATTEST
RECALL
```

An operation is effect-safe only if:

$$\text{admitted_effect} \leq \text{warranted_effect}$$

If the relationship is unknown, the operation must refuse or emit a gap certificate.

8.4 Receipt

A receipt is evidence of a transition, refusal, gap, or standing change.

```
Receipt =
  (receipt_id,
   role,
   operation_id,
   canonical_operation_hash,
   state_prev_hash,
   state_next_hash,
   content_hash,
   semantic_hash,
   jurisdiction_hash,
   authority_context_hash,
   effect_class,
   result_kind,
   timestamp_policy,
   witness_refs,
   signature_status,
   provenance_refs)
```

Receipt presence alone is not enough. The receipt must bind the correct operation, role, effect, authority context, state, and result kind.

8.5 Typed Claim and Nonclaim

A typed claim is a registry-bound assertion.

```
TypedClaim =
  (claim_id,
   claim_type_id,
   subject_ref,
   subject_kind,
   predicate_type_id,
   object_ref,
   object_kind,
   scope_kind,
   standing_level,
   evidence_refs,
   nonclaim_refs_checked,
   entailment_rule_id,
   entailment_status)
```

A nonclaim is a forbidden inference.

```
Nonclaim =
  (nonclaim_id,
   forbidden_inference,
```

```
  preserved_by,
  retirement_condition,
  scope,
  standing_ceiling)
```

Nonclaims prevent evidence from being used to imply more than it supports.

8.6 Standing

Standing is the level of admissibility earned by evidence.

```
NONE
FIXTURE
CONFORMANCE
REFERENCE
EXPERIMENTAL
RELEASE
PRODUCTION
NORMATIVE
```

Standing is never implied. It is admitted by a governed transition.

8.7 Gap Certificate

A gap certificate records unsupported or partially preserved semantics.

```
GapCertificate =
  (gap_id,
   source_operation,
   target_layer,
   missing_semantics,
   unsupported_effects,
   unsupported_receipts,
   blocked_claims,
   safe_fallback)
```

Gap certificates are how GSOL avoids silent semantic loss.

9. Claims, Testimony, Evidence, and Standing

GSOL distinguishes four often-confused objects.

```
Testimony: output asserted by a model, tool, actor, or process.
Evidence: a record that may support a typed claim.
Claim: a typed assertion admitted under an entailment rule.
Standing: a governed level of admissibility assigned to a claim or
artifact.
```

The minimal inference discipline is:

Testimony production.

$$\text{ModelInvoke}(m, x) \rightarrow \text{Testimony}(t)$$

Evidence binding.

$$\begin{aligned} &\text{Receipt}(r) \\ &\wedge \text{binds}(r, t) \\ &\rightarrow \text{Evidence}(e) \end{aligned}$$

Claim candidate formation.

$$\begin{aligned} &\text{Testimony}(t) \\ &\wedge \text{Classify}(t, p) \\ &\rightarrow \text{ClaimCandidate}(c) \end{aligned}$$

Claim admission.

$$\begin{aligned} &\text{Evidence}(e) \\ &\wedge \text{Typed}(c) \\ &\wedge \text{Entails}(e, c, \rho) \\ &\wedge \text{NonclaimsChecked}(c) \\ &\rightarrow \text{AdmittedClaim}(c) \end{aligned}$$

Standing admission.

$$\begin{aligned} &\text{AdmittedClaim}(c) \\ &\wedge \text{StandingRule}(c, \sigma) \\ &\rightarrow \text{Standing}(c, \sigma) \end{aligned}$$

The following inferences are invalid:

$$\text{Testimony}(t) \not\Rightarrow \text{AdmittedClaim}(c)$$

$$\text{Evidence}(e) \not\Rightarrow \text{Authority}(a)$$

$$\text{Receipt}(r) \not\Rightarrow \text{Standing}(c, \sigma)$$

$$\text{SuccessfulRun}(x) \not\Rightarrow \text{ProductionReady}(x)$$

A minimal entailment relation is:

$$\text{Entails}(\text{EvidenceSet}, \text{TypedClaim}, \text{RuleProfile}) \rightarrow \text{EntailmentStatus}$$

where:

```
EntailmentStatus =
  SUPPORTED
  NOT_SUPPORTED
  CONTRADICTED
  UNDECIDABLE
  OUT_OF_SCOPE
  GAP_REQUIRED
```

GSOL does not claim that arbitrary natural-language entailment is decidable. It requires that the machine-bearing claim position be typed, registry-bound, and checked under an explicit rule profile.

10. Refusal as Successful Governance

In ordinary programming, refusal may look like an error. In GSOL, refusal is a first-class successful result when it preserves a boundary that should not be crossed.

A refusal is required when:

```
authority is missing
authority is stale
jurisdiction mismatches
effect exceeds authority
input is invalid
state boundary mismatches
receipt obligation cannot be satisfied
claim is unsupported
nonclaim would be violated
requested standing exceeds evidence
lowering target cannot preserve semantics
temporal validity fails
revocation or recall blocks use
merge conflict remains unresolved
```

A refusal result has the form:

$$\text{Refuse}(c, \mathcal{R}_{\text{refusal}}, \Delta_0)$$

The refusal receipt may support the claim that the system preserved a constitutional boundary. It does not support the claim that the requested operation

completed. This is why refusal is a success condition in GSOL: it demonstrates that the system preserved admissibility boundaries instead of converting insufficient authority, evidence, or semantics into an unearned effect.

11. Syntax Sketch

A complete GSOL grammar should be small, regular, parseable, and canonical. An illustrative grammar is:

```
program         ::= metadata? import* operation+
metadata        ::= "gsol" version_decl profile_decl? current_head_decl?
import          ::= "import" qualified_id
operation       ::= "operation" id "{" op_body "}"

op_body         ::= version_decl
                    jurisdiction_decl
                    profile_decl?
                    inputs_decl
                    outputs_decl?
                    authority_decl
                    warrants_decl?
                    effects_decl
                    requires_decl
                    refuses_decl
                    admits_decl
                    emits_decl
                    claims_decl?
                    nonclaims_decl?
                    standing_decl
                    temporal_decl?
                    debts_decl?
                    lowerings_decl
                    proofs_decl?

version_decl    ::= "version" string
jurisdiction_decl ::= "jurisdiction" qualified_id
profile_decl    ::= "profile" qualified_id
current_head_decl ::= "current_head" qualified_id

inputs_decl     ::= "inputs" block(field_decl*)
outputs_decl    ::= "outputs" block(field_decl*)
field_decl      ::= id ":" type_ref

authority_decl ::= "authority" block(predicate*)
warrants_decl  ::= "warrants" block(warrant_ref*)
effects_decl   ::= "effects" block(effect_stmt*)
```

```
requires_decl  ::= "requires" block(predicate*)
refuses_decl   ::= "refuses" block(refusal_rule+)
admits_decl    ::= "admits" block(action+)
emits_decl     ::= "emits" block(receipt_decl+)
claims_decl    ::= "claims" block(typed_claim_stmt*)
nonclaims_decl ::= "nonclaims" block(nonclaim_stmt*)
standing_decl  ::= "standing" block(standing_stmt+)
temporal_decl  ::= "temporal" block(temporal_stmt*)
debts_decl     ::= "debts" block(debt_stmt*)
lowerings_decl ::= "lowerings" block(lowering_stmt+)
proofs_decl    ::= "proofs" block(proof_stmt*)

refusal_rule   ::= predicate_ref "->" refusal_code
receipt_decl   ::= id ":" "Receipt" "(" receipt_attrs ")"
lowering_stmt  ::= target_id lowering_status
lowering_status ::= "supported" | "requires_gap_certificate" |
"not_claimed"
```

The grammar prefers explicit blocks over clever shorthand. Governed systems should be legible to auditors.

12. Worked Example: Governed Model Invocation

This section gives an end-to-end example: proposal, admission, effect, receipt, claim and nonclaim update, and lowering.

12.1 Scenario

A user asks a governed AI system:

```
Summarize EvidenceBundle E under strict evidence profile.
```

The system proposes to invoke a model. The model may summarize evidence, but it may not admit new factual claims, call tools, write memory, or publish externally.

12.2 GSOL Operation

```
operation governed_model_summary {
  version "0.1"
  jurisdiction AOK
  profile strict_evidence

  inputs {
    request: RequestEnvelope
    evidence: EvidenceBundle
    context: SessionContext
    warrant: ModelInvocationWarrant
```

```
  reactor: ReactorRef
}

outputs {
  testimony: ModelTestimony
  summary_candidate: ResponseCandidate
}

authority {
  requires warrant.permits(MODEL_INVOKE)
  requires warrant.jurisdiction == jurisdiction
  requires warrant.temporal_status == ACTIVE
}

effects {
  requested MODEL_INVOKE
  admitted MODEL_INVOKE
  forbidden [TOOL_CALL, MEMORY_WRITE, CLAIM_ADMIT, PUBLISH]
}

requires {
  request.canonical == true
  evidence.hash_bound == true
  context.profile == strict_evidence
  reactor.kind in [DeterministicFixtureReactor, ReplayReactor,
  LocalModelReactor]
}

refuses {
  missing_warrant -> REFUSE.MISSING_WARRANT
  stale_warrant -> REFUSE.STALE_WARRANT
  evidence_unbound -> REFUSE.INVALID_INPUT
  policy_downgrade -> REFUSE.POLICY_DOWNGRADE
  effect_exceeds_authority -> REFUSE.EFFECT_EXCEEDS_AUTHORITY
}

admits {
  reactor.invoke(request, evidence, context)
}

emits {
  pre_model_receipt: Receipt(role="pre_model_invocation")
  closeout_receipt: Receipt(role="model_closeout")
  refusal_receipt: Receipt(role="refusal")
}
```

```
  claims {
    supports CLAIM.MODEL_INVOCATION_WITNESSED
  }

  nonclaims {
    preserves NONCLAIM.MODEL_OUTPUT_IS_NOT_TRUTH
    preserves NONCLAIM.MODEL_OUTPUT_IS_NOT_CLAIM_ADMISSION
    preserves NONCLAIM.MODEL_OUTPUT_IS_NOT_MEMORY
    preserves NONCLAIM.MODEL_OUTPUT_IS_NOT_AUTHORITY
  }

  standing {
    may_support CONFORMANCE
    cannot_promote PRODUCTION
  }

  lowerings {
    deterministic_fixture supported
    python_reference supported
    mcir requires_gap_certificate
    binary not_claimed
  }

  proofs {
    warrant_check required
    receipt_binding required
    testimony_classification required
    negative_fixture required
  }
}
```

12.3 Mechanical Coverage of Refusal Receipts

Each declared refusal path is covered by the `refusal_receipt` obligation.

The coverage map is:

```
missing_warrant -> REFUSE.MISSING_WARRANT -> receipt role: refusal
stale_warrant -> REFUSE.STALE_WARRANT -> receipt role: refusal
evidence_unbound -> REFUSE.INVALID_INPUT -> receipt role: refusal
policy_downgrade -> REFUSE.POLICY_DOWNGRADE -> receipt role: refusal
effect_exceeds_authority -> REFUSE.EFFECT_EXCEEDS_AUTHORITY ->
receipt role: refusal
```

A checker can therefore verify a simple coverage condition:

$$\forall r \in \mathrm{ref}(O),\ \exists e \in \mathrm{emit}(O) : \mathrm{role}(e) = \mathrm{refusal}$$

A stronger profile may require a distinct refusal receipt template per refusal code. The v0.1 rule requires at least one declared refusal receipt role for every refusal path.

12.4 Transition Walkthrough

Proposal. The system proposes `MODEL_INVOKE` under `strict_evidence`.

Admission. The warrant is active, the evidence bundle is hash-bound, and the requested effect is limited to model invocation. Admission succeeds.

Effect. The reactor is invoked. The output is `ModelTestimony`.

Receipt. The system emits a pre-model receipt and a closeout receipt. If the warrant were missing or stale, it would emit a refusal receipt instead.

Claim update. The operation may support only `CLAIM.MODEL_INVOCATION_WITNESSED`. It does not admit the factual content of the model output.

Nonclaim preservation. The system preserves the nonclaims that model output is not truth, not claim admission, not memory, and not authority.

Lowering. The operation may lower to deterministic fixture or Python reference. MCIR lowering requires a gap certificate because the current target cannot yet represent all testimony classification and nonclaim preservation semantics directly. Binary lowering is not claimed.

12.5 Refusal Variant

If the warrant is missing, the transition is:

$$T(S, I, A) = \text{Refuse}(\text{REFUSE.MISSING_WARRANT}, \mathcal{R}_{\text{refusal}}, \Delta_0)$$

This is not an execution failure. It is a successful governed refusal.

13. Lowering Example with Gap Certificate

Suppose the operation above is lowered to a Python reference harness. The Python target can preserve:

```
warrant check
effect check
reactor invocation
pre-model receipt
closeout receipt
refusal receipt
```

But the target cannot natively preserve:

```
typed claim entailment
semantic hash equivalence
MCIR-level lowering proof
binary realization proof
```

The lowering result is therefore not a full preservation claim. It is:

```
LoweringResult =
  LoweredArtifact(python_reference)
  + GapCertificate(
      gap_id="GAP.MODEL_SUMMARY.MCIR_TYPED_CLAIM",
      source_operation="governed_model_summary",
      target_layer="mcir",
      missing_semantics=[
        "typed_claim_entailment",
        "semantic_hash_equivalence",
        "nonclaim_preservation_proof"
      ],
      unsupported_effects=[],
      unsupported_receipts=[],
      blocked_claims=[
        "CLAIM.UNBOUNDED_GSOL_TO_MCIR_LOWERING",
        "CLAIM.BINARY_REALIZATION"
      ],
      safe_fallback="python_reference_with_receipts"
    )
```

The gap certificate prevents a partial lowering from being overclaimed as full semantic preservation. The example also illustrates a general lowering rule:

$$\text{loses_required_semantics}(\text{source}, \text{target}) \Rightarrow \text{LoweringResult} = \text{LoweredArtifact}(\text{target}) + \text{GapCertificate}(g)$$

Silent semantic loss is not a valid lowering result.

14. Threat and Adversary Model

GSOL is designed for systems containing honest components, confused components, stale components, and malicious components.

Adversary classes include:

```
lying_model
malicious_tool_provider
compromised_coder
tampered_archive
defective_gate
stale_current_head_surface
```

```
malicious_or_model_based_classifier
colluding_or_looping_agents
policy_downgrade_actor
```

GSOL primarily defends against semantic governance failures:

```
authority spoofing
effect laundering
receipt role collapse
claim overpromotion
nonclaim erasure
jurisdiction confusion
registry substitution
current-head drift
lowering misrepresentation
semantic hash drift
pseudocode ambiguity
capability-authority confusion
fixture-production collapse
silent refusal removal
stale evidence promotion
tool-output authority laundering
model-testimony truth laundering
memory-write laundering
```

GSOL does not by itself defeat all adversaries. In particular, same-tree verification proves internal consistency only. If a compromised archive contains its own mutated gates and receipts, internal verification may be insufficient. External trust anchors are required for provenance claims.

Internal consistency means that an artifact agrees with its own governed records: manifests match files, receipt references resolve, current-head metadata is coherent, and verification vectors pass under the included rules. External attestation means that an external witness, signature, transparency log, independent rebuild, timestamp authority, or trusted baseline digest supports a claim about origin, time, producer, or non-tampering. GSOL can represent both surfaces, but it must not collapse them. An internally consistent artifact is not automatically externally attested.

Root-of-trust surfaces include:

```
trusted_external_baseline_digest
governance_current_head
source_manifest
detached_evidence_manifest
full_verification_vector
governor_review_surface
external signature or transparency log, when available
```

15. Canonicalization, Hashing, and Implementation Feasibility

GSOL is feasible to implement incrementally because its first-order requirements are ordinary language-tooling tasks:

```
parse a structured grammar
build a canonical AST
normalize identifiers and field order
serialize deterministically
compute operation hashes
bind registry references
run type checks
run effect checks
run refusal coverage checks
verify receipt obligations
emit gap certificates
```

A canonical operation hash is computed over normalized semantic content:

$$\text{operation_hash} = \text{HASH}(\text{canonical_operation_payload})$$

Canonicalization excludes comments from semantic hash and includes all governing fields. This enables drift detection and registry verification.

A minimal implementation need not execute GSOL. It can begin as a parser, canonicalizer, checker, and renderer. Runtime interpretation, MCIR lowering, and live AI harness integration can be layered later.

16. Comparison Matrix

The comparison is stated as compact prose rather than a wide table, because the relevant contrast is qualitative rather than numerical.

Compiler IR. Compiler intermediate representations support program transformation and code generation. They usually preserve control flow, data flow, types, and operations. They usually externalize authority, refusal, standing, and nonclaims. GSOL preserves governed operation meaning before lowering.

Policy language. Policy languages support authorization decisions. They usually preserve rules and permissions. They usually externalize receipts, claims, lowering, and standing. GSOL represents the whole transition, not only an allow-or-deny decision.

Provenance system. Provenance systems support origin and derivation tracking. They usually preserve agents, activities, and entities. They usually externalize admission semantics, nonclaims, and refusal paths. GSOL binds provenance to admissible operation semantics.

Supply-chain attestation. Supply-chain attestation systems support artifact integrity and build provenance. They usually preserve materials, builders, and steps. They usually externalize semantic claim boundaries. GSOL specifies what the attested operation means.

AI harness. AI harnesses support runtime orchestration of model calls, tool calls, and memory flow. They usually externalize canonical typed operation semantics. GSOL supplies the operation contract that harnesses can enforce.

Pseudocode. Pseudocode supports human explanation. It usually preserves intent and readability. It usually externalizes canonicality, evidence, standing, and refusal. GSOL is canonical and checkable, not merely explanatory.

Natural-language policy. Natural-language policy supports human governance. It usually preserves normative intent. It usually externalizes machine-checkable operational structure. GSOL turns policy-relevant operations into typed artifacts.

17. Current Head, Registry Dependence, and Versioning

A GSOL archive should not scatter its current version, current milestone, standing, next rung, and release profile across many hand-maintained surfaces. Current-head identity should be governed from one source.

A current-head object should bind:

```
package_name
package_version
current_milestone
current_standing
current_claim_id
predecessor_milestone
next_allowed_rung
release_profile
source_archive_profile
detached_evidence_bundle_hash
verification_vector_ref
```

Registries are also central. GSOL depends on stable registries for effects, refusal codes, receipt roles, typed claims, predicates, nonclaims, standing levels, jurisdictions, warrants, lowering targets, types, temporal policies, and proof obligations.

This registry dependence is both a strength and a limitation. It makes operations checkable, but registry drift must be detected and migrated through admissible records.

18. Limitations

GSOL v0.1 is a pre-release specification. Its limits include:

```
full parser is not yet normative
full typed-claim migration is not complete
full entailment engine is not implemented
arbitrary natural-language entailment remains undecidable
runtime execution is not implemented
live AI harness integration is not implemented
streaming response semantics are not complete
production attestation is not complete
automatic merge runtime is not complete
adaptive receipt-tiering runtime is not complete
unbounded GSOL-to-MCIR lowering is not complete
unbounded GSOL-to-Jnana lowering is not complete
native or binary execution is not claimed
```

The most important theoretical limit is entailment. If GSOL allowed arbitrary free prose as the machine-bearing claim position, sound checking would collapse. GSOL therefore requires typed, registry-bound claims for machine admission, while allowing prose as explanation.

The second major limit is registry dependence. A typed semantic language depends on stable vocabularies. Drift, migration, and versioning must therefore be governed.

The third major limit is partial lowering. A target substrate may preserve executable behavior while losing authority, refusal, receipt, claim, nonclaim, or standing semantics. GSOL treats this as a gap rather than an equivalence claim.

The fourth major limit is non-universality. GSOL is not meant to express arbitrary computation. Its competence is measured by whether it can represent admissibility-relevant transitions, not whether it can replace executable languages.

The fifth major limit is external trust. Internal receipts and verification vectors can show consistency, but provenance claims require external anchors such as signatures, transparency logs, independent witnesses, or trusted baseline digests.

In short, GSOL v0.1 is bounded by undecidable entailment, partial lowering, registry dependence, non-universality, and the distinction between internal consistency and external attestation. These are not incidental weaknesses; they are the boundaries that keep GSOL from becoming an overclaiming runtime or truth engine.

19. Reproducibility and Artifact Statement

This paper presents a conceptual language design. Reproducibility for the concept consists of checking whether future GSOL artifacts can be parsed, canonicalized,

hashed, type-checked, and evaluated against positive and negative conformance fixtures. A reference artifact may include a grammar, canonical AST schema, registry set, example operations, verifier scripts, receipt fixtures, negative fixtures, and gap certificates.

Claims in this paper should be read as design claims unless explicitly attached to an implementation artifact. GSOL v0.1 defines a direction and a conformance boundary; it does not claim production runtime completeness.

20. Discussion: Why GSOL Should Not Be Turing-Complete

A natural question is whether GSOL should be powerful enough to express arbitrary computation. The answer is no.

If GSOL became a general-purpose programming language, the governance layer would become another execution surface. Computation could hide inside the language intended to constrain computation. This would undermine the separation between semantic governance and executable realization.

The better goal is constitutional adequacy. GSOL should express every transition whose admissibility depends on authority, effect, refusal, receipt, claim, standing, time, provenance, or lowering. Arbitrary algorithms should live in executable substrates. GSOL should govern what those algorithms are allowed to mean and do.

A maximally competent GSOL is therefore:

```
a decidable, typed, compositional governed semantic operation language
that can express every constitutionally meaningful transition,
bind each transition to authority, evidence, effect, refusal, and
receipt semantics,
and lower honestly into executable substrates without becoming the
substrate itself.
```

21. Conclusion

GSOL is proposed as a language of admissible operation: a governed semantic operation language for representing computational transitions before execution.

Its contribution is not a new syntax for ordinary programming. Its contribution is a semantic layer that preserves distinctions ordinary computational systems often erase: capability versus authority, testimony versus claim, evidence versus standing, receipt versus provenance, execution versus admission, and correspondence versus equivalence.

The need for such a layer becomes acute in systems where AI models, tools, memory, claims, evidence, and executable artifacts interact. In those systems,

the danger is not only that code may fail. The deeper danger is that a system may overinterpret its own success.

GSOL is designed to resist that overinterpretation.

In plain language: GSOL is not the machine. It is the oath the machine must keep before it moves.

Corpus-Wide Reference Positioning

This paper argues that computational systems increasingly perform actions whose significance cannot be determined from execution alone: a program may run, a model may answer, a tool may return output, or an artifact may be built without establishing whether the transition was authorized, evidenced, bounded, or admissible. Its nearest external comparison fields are formal, token, semantic. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. Statistical compression can improve efficiency while destroying distinctions needed for interpretation; semantic labels can conversely overstate stability that usage does not support. Formal vocabularies and graph links make commitments explicit but cannot by themselves establish correct identity, shared interpretation, or lossless translation between contexts. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] George C. Necula. "Proof-Carrying Code." *POPL '97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E2**] Xavier Leroy. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

[**E3**] Patrick Cousot and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *POPL '77*, 1977. DOI: 10.1145/512950.512973. https://doi.org/10.1145/512950.512973 **Relationship:** Predecessor formal framework for sound approximation and fixpoint reasoning.

[**E4**] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. https://lamport.azurewebsites.net/tla/book.html **Relationship:** Predecessor specification discipline for state machines, safety, liveness, and refinement.

[**E5**] Rico Sennrich, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units." *ACL 2016*, pp. 1715-1725. DOI: 10.18653/v1/P16-1162. https://doi.org/10.18653/v1/P16-1162 **Relationship:** Predecessor subword tokenization method used as a contrast to semantically governed token boundaries.

[**E6**] Taku Kudo and John Richardson. "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing." *EMNLP 2018: System Demonstrations*. DOI: 10.18653/v1/D18-2012. https://doi.org/10.18653/v1/D18-2012 **Relationship:** Predecessor tokenizer implementation used to separate statistical segmentation from semantic admissibility.

[**E7**] Peter D. Grunwald. *The Minimum Description Length Principle*. MIT Press, 2007. mitpress.mit.edu source **Relationship:** Predecessor theory of compression and model selection; compression is not assumed to preserve governance or meaning.

Ableman corpus

[**A1**] Adam Ableman. *MCIR: A Canonical Intermediate Representation for Governed Transformation*. Ableman Corpus current edition; work identity `PAPER.PUB.21095303`;
source SHA-256
`EA129ECBA95F7B07CAA7B33A54F91F52913C94A14DA227EB9A06CFD083E4E5BF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Governed Semantic Compilation*. Ableman Corpus current edition; work identity `PAPER.PUB.18453681`;
source SHA-256
`5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Hybrid Semantic Compiler Architectures*. Ableman Corpus current edition; work identity `PAPER.PUB.21228038`;
source SHA-256
`EBCD6DDDA9B547639AEBED79FB68069473CF1248E41C527B48C9624BBE2C0496`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Admissibility Engineering*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-F1E33B5E973B0900`;
source SHA-256
`CDC80271268802CA5CB4DE3624B992FB2FF07D677BF945C4B349BD307C4C2A6E`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Everything Is a Compiler*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-30FB8F95BD990A47`;
source SHA-256
`FE0F88A1F125317285BE4378E5F27ACC9FEB83ADE3FF329A3DA9F9CB026DF9A9`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Governed Semantic Tokens: Tokenization as an Admissibility Boundary*. Ableman Corpus current edition; work identity `PAPER.PUB.18469507`;
source SHA-256
`5C44175B87C514892A0DC9743E09D03893E970AF5419F2E6D9171651BE2A9A54`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Token Parsimony*. Ableman Corpus current edition; work

identity PAPER.PUB.20382825;
source SHA-256
EC3AA22D32E2BBA21CFCB6D1FA2AE6EA8AC11B7781D19C51530A2F6D924B2E91.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

LLVM Project. "LLVM Language Reference Manual." https://llvm.org/docs/LangRef.html

W3C. "PROV-O: The PROV Ontology." https://www.w3.org/TR/prov-o/

Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. "in-toto: Providing farm-to-table guarantees for bits and bytes." USENIX Security, 2019. usenix.org source

in-toto Project. "in-toto: A framework to protect supply chain integrity." https://in-toto.io/

SLSA Project. "Supply-chain Levels for Software Artifacts." https://slsa.dev/

SLSA Project. "Provenance." https://slsa.dev/spec/v1.0/provenance

---

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.21214886